

Week 2 — live coding (completed)

Social and Cultural Analytics (7AAVBCS5)

The filled-in version of what we typed in the lecture, released afterwards so you can check what you missed.

1. Lists

```
ages = [18, 21, 30, 24, 22, 22]
ages
```

```
[18, 21, 30, 24, 22, 22]
```

```
print("how many:", len(ages))
print("first:   ", ages[0])
print("third:   ", ages[2])
print("last:    ", ages[-1])
```

```
how many: 6
first:    18
third:    30
last:     22
```

Positions start at zero

The first item is at 0. The last is at `len(ages) - 1`, which is 5 here — not 6.

```
# There is no position 6, so this raises IndexError.
# print(ages[6])
```

Slicing — the end is excluded

```
print(ages[0:3]) # positions 0, 1, 2 -- three items
print(ages[:2])  # from the start
print(ages[3:])  # to the end
```

```
[18, 21, 30]
[18, 21]
[24, 22, 22]
```

Changing a list

```
cities = ["London", "Manchester", "Rome"]
cities.append("Lima")
cities[0] = "Edinburgh"
print(cities)
```

```
['Edinburgh', 'Manchester', 'Rome', 'Lima']
```

2. The trap: lists do not do maths

```
# This REPEATS the list. It does not double the numbers, and there is no error.
ages * 2
```

```
[18, 21, 30, 24, 22, 22, 18, 21, 30, 24, 22, 22]
```

3. Arrays do

```
import numpy as np

ages_array = np.array(ages)
print(ages_array * 2)
```

```
[36 42 60 48 44 44]
```

```
temperatures_c = np.array([20, 25, 18, 30])
temperatures_f = (temperatures_c * 9 / 5) + 32
print(temperatures_f)
```

```
[68.  77.  64.4 86. ]
```

```
# A comparison on an array gives one True/False per element.
# This is where filtering data frames comes from in Week 3.
print(temperatures_c > 25)
```

```
[False False False  True]
```

Arrays force one type

```
# Everything becomes text: the order is text > number > True/False.
print(np.array([1, 2, True, "word"]))
```

```
['1' '2' 'True' 'word']
```

```
# With no text present, True and False become 1 and 0.
print(np.array([True, False, 1]))
```

```
[1 0 1]
```

4. Describing data

```
print("count  ", len(ages_array))
print("min    ", ages_array.min())
print("max    ", ages_array.max())
print("sum    ", ages_array.sum())
print("mean   ", ages_array.mean())
print("median ", np.median(ages_array))
print("std    ", ages_array.std())
```

```
count    6
min      18
max      30
sum      137
mean     22.833333333333332
median   22.0
std      3.670452590924207
```

Most of these are methods on the array — `ages_array.mean()`. `np.median` is the odd one out and has to be called as a function.

`.std()` is the population standard deviation. R's `sd()` is the sample one; for that, use `.std(ddof=1)`.

5. Building text with f-strings

```
year = 1945
print(f"WW2 ended in {year} when the Allies won.")
```

WW2 ended in 1945 when the Allies won.

```
print(f"The mean age is {ages_array.mean():.1f}")
```

The mean age is 22.8

`:.1f` rounds to one decimal place. This replaces R's `paste()` and `paste0()`.

6. Dictionaries

```
student = {"name": "Sarah", "age": 21, "enrolled": True}
print(student["name"])

student["courses"] = ["History", "Stats"]
print(student.keys())
```

```
Sarah
dict_keys(['name', 'age', 'enrolled', 'courses'])
```

A list finds things by position; a dictionary finds them by name. In Week 3 a row of a data frame behaves much like a dictionary.

7. Making a decision

```
temperature = 20

if temperature > 10:
    print("go for a run")
else:
    print("stay at home")
```

go for a run

```
mark = 65

if mark >= 70:
    print("First")
elif mark >= 60:
    print("2:1")
elif mark >= 50:
    print("2:2")
else:
    print("Fail")
```

2:1

Order matters: Python stops at the first branch that is true. If the `>= 50` test came first, a mark of 80 would print “2:2”.

```
year = 1560

if year >= 1400 and year <= 1780:
    print("Early modern")
```

Early modern

and, or and not are words in Python, where R used `&&` and `||`.

8. How not to lose an afternoon

```
secret = "I was defined in a cell that no longer exists"
```

```
# In the lecture we deleted the cell above and this still worked, because the variable  
# lives in memory rather than in the notebook. Restart and Run All, and it breaks.  
print(secret)
```

I was defined in a cell that no longer exists

The habit: before you close a notebook, or submit one, use **Restart and Run All**. If it does not survive that, it depends on something you have since changed or deleted — and it will not work for anyone else, including a marker.

End of live coding.